

Deep Reinforcement Learning

Matteo Hessel

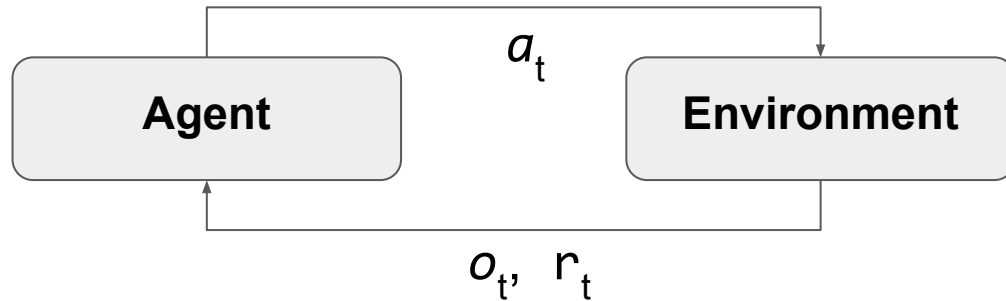
The only stupid question is the one you never asked.

Rich Sutton

1. Introduction

The reinforcement learning problem

A learning system (the **agent**) must learn to act in the universe it's embedded in (the **environment**) to maximize a scalar feedback signal (the **reward**).



Policies

The agent's behaviour is defined by a **policy** $\pi(A_t/S_t)$, i.e. by a probability distribution over actions, conditioned on the current **state**

The agent's objective is to find an **optimal** policy $\pi^*(A_t/S_t)$ that maximises the **return**, i.e. the sum of discounted rewards collected from any state S_t onwards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

the **discount** γ modulates the relative weight of immediate vs distant rewards.

Values

If the environment or the agent is **stochastic**, the return is a random variable, thus the objective is typically formulated as maximizing **expected returns**, or **values**.

$$V(s) = E\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s\right] \rightarrow \textit{state value}$$

$$Q(s, a) = E\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a\right] \rightarrow \textit{state-action value}$$

The expectations are over both the environment and agent stochasticity.

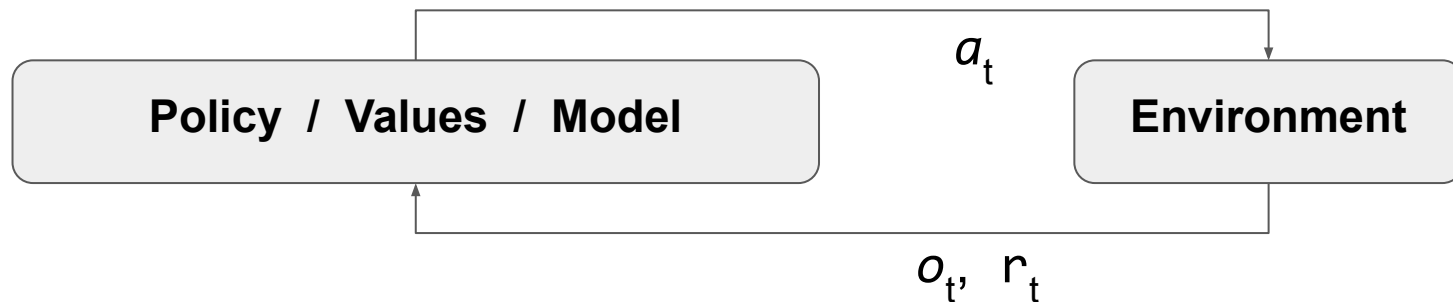
Models

In RL we often assume that we do not know “a priori” the environment dynamics:

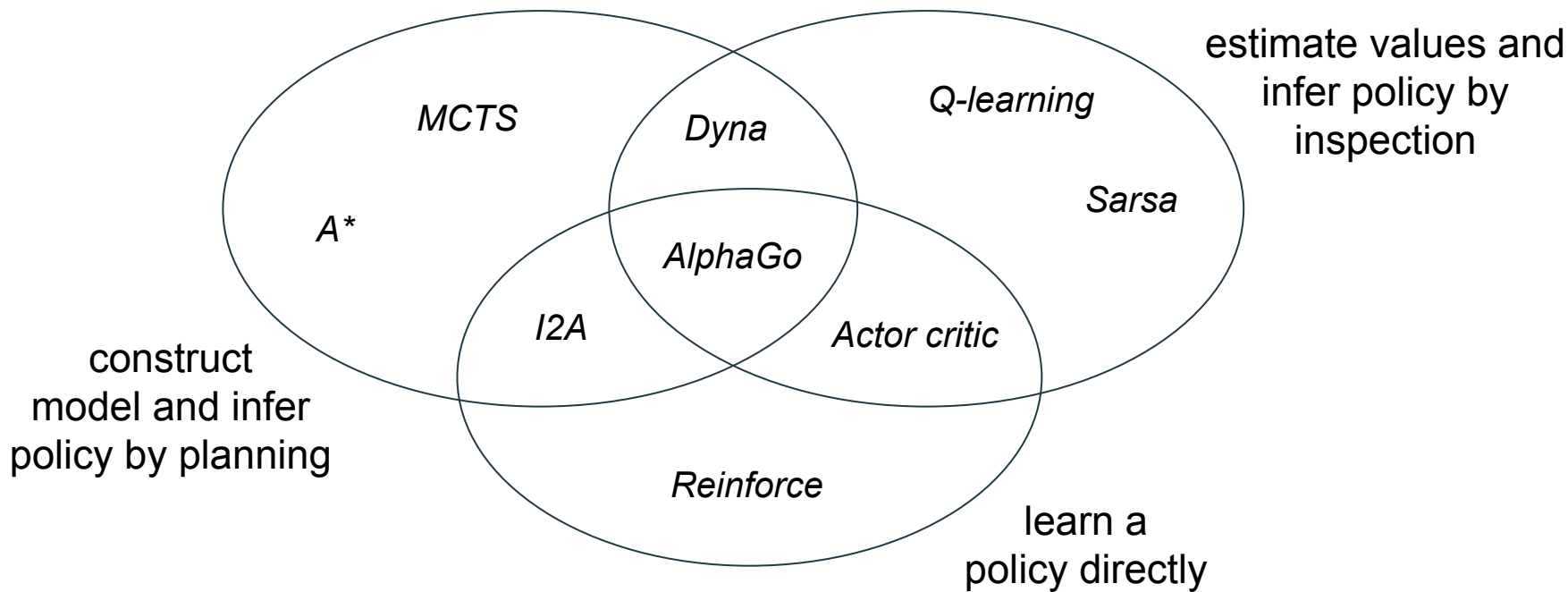
- values and policies may be learned **model-free**, directly from interaction
- or we may use those same interactions to learn a **model** of the environment

RL solutions: the Big Picture

- Learn **policy** directly - **execute** it
- Learn **values** of each action - infer policy by **inspection**
- Learn a **model** - infer policy by **planning**



RL solutions: the Big Picture



Deep Reinforcement Learning

- *Policy* $\pi : S_t \rightarrow p(A_t)$
- *Value* $v : S_t \rightarrow V(S_t)$
- *Model* $m : S_t, A_t \rightarrow S_{t+1}, R_{t+1}$

Ultimately these are just **functions**,

and we need a flexible way of **representing** and **learning** them

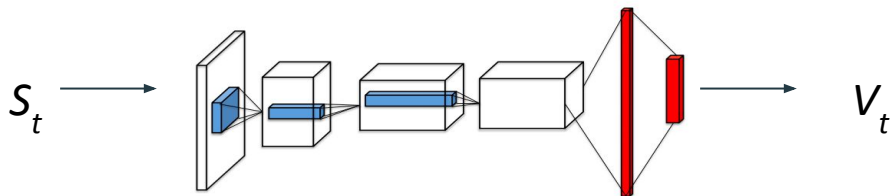
Function approximation

Consider the problem of representing and learning state values V :

A simple choice could be to represent these as a linear function of the state features:

$$v_w(s) = w^T s + b$$

Deep reinforcement learning focuses on the case where the function is represented by a large neural network, whose parameters are trained via **gradient descent** on a suitable loss function



Why Deep Learning?

Deep learning is not the only answer, but it's a pretty convenient one

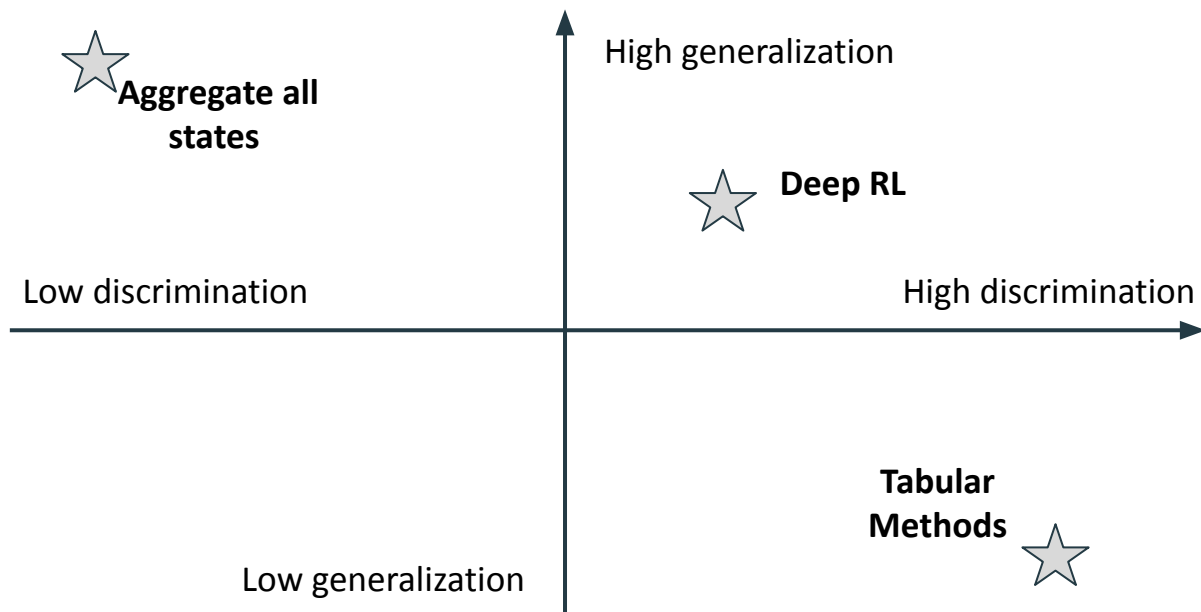
- **Table look-up?**

- Learn about each state separately
- No **generalization**, does not scale to large state spaces

- **Linear?**

- $y_t = \mathbf{w}^T S_t + b$
- Can work very well if you have a rich state representation S_t
- Hard-coding a suitable **rich state representation** S_t is difficult!

Why Deep Learning?



Credits: Adam White

MC prediction with deep nets

Consider now the **prediction** problem of estimating state values for a fixed policy π :

Lets approximate the values using a **neural network** $v_{\theta}(s)$

with parameters θ , state features S as inputs, and a single scalar output.

MC prediction with deep nets

Consider now the **prediction** problem of estimating state values for a fixed policy π :

Lets approximate the values using a **neural network** $v_{\theta}(s)$

with parameters θ , state features S as inputs, and a single scalar output.

Deep Monte Carlo Prediction trains the parameters of the network to minimize:

$$L = (G_t - v_{\theta}(S_t))^2$$

MC prediction with deep nets

Consider now the **prediction** problem of estimating state values for a fixed policy π :

Lets approximate the values using a **neural network** $v_{\theta}(s)$

with parameters θ , state features S as inputs, and a single scalar output.

Deep Monte Carlo Prediction trains the parameters of the network to minimize:

$$\begin{aligned} L &= (G_t - v_{\theta}(S_t))^2 \\ \Delta\theta &= -\nabla_{\theta} L = (G_t - v_{\theta}(S_t)) \nabla_{\theta} v_{\theta}(S_t) \end{aligned}$$

Gradient descent

TD(0) prediction with deep nets - I

Monte Carlo prediction is unbiased, but high variance

TD(0) prediction with deep nets - I

Monte Carlo prediction is unbiased, but high variance

What would a deep **temporal difference** algorithm look like?

Deep TD(0) Prediction approximates the Monte Carlo update:

$$\Delta\theta = (G_t - v_\theta(S_t)) \nabla_\theta v_\theta(S_t) \quad [\text{MC}]$$

TD(0) prediction with deep nets - I

Monte Carlo prediction is unbiased, but high variance

What would a deep **temporal difference** algorithm look like?

Deep TD(0) Prediction approximates the Monte Carlo update:

$$\begin{array}{l} \Delta\theta = (G_t - v_\theta(S_t)) \nabla_\theta v_\theta(S_t) \quad [\text{MC}] \\ \Delta\theta = (\underline{R_t + \gamma v_\theta(S_{t+1})} - v_\theta(S_t)) \nabla_\theta v_\theta(S_t) \quad [\text{TD}] \end{array}$$

TD(0) prediction with deep nets - II

What objective function is TD(0) optimizing?

→ This update is actually **not the gradient** of any objective function

We can still implement it as a loss in the context of common DL frameworks?

$$L = (|R_t + \gamma v_{\theta}(S_{t+1})| - v_{\theta}(S_t))^2$$

By using the **stop gradients** operator to ignore the dependence of the loss on $v_{\theta}(S_{t+1})$

Q-learning with deep nets

The same approach extends also to **control**.

In this case we strive to estimate the action values $q_{\theta}(s, a)$ of the optimal policy.

Q-learning with deep nets

The same approach extends also to **control**.

In this case we strive to estimate the action values $q_{\theta}(s, a)$ of the optimal policy.

The neural network will now have to output a vector, with a prediction per action

Q-learning with deep nets

The same approach extends also to **control**.

In this case we strive to estimate the action values $q_{\theta}(s, a)$ of the optimal policy.

The neural network will now have to output a vector, with a prediction per action

A deep Q-learning update will then look very much alike the previous one

$$\Delta\theta = (R_t + \gamma \max_{a'} q_{\theta}(S_{t+1}, a') - q_{\theta}(S_t, A_t)) \nabla q_{\theta}(S_t, A_t)$$

Experience Replay

Compute the Q-learning loss not on the most recent transition in the environment but instead on an old one sampled from a **large memory buffer**

- The larger the buffer, the less correlated consecutive updates
- We can reuse transitions in multiple updates
- We can **prioritize** novel by sampling more often “surprising” data from which there is much to learn
- We can compute updates from batches rather than single samples

3. Generalization in Deep RL

Generalization

Generalization is the process of updating your beliefs about one or more states in response to observing novel information about a *different* set of states

Generalization

Generalization is the process of updating your beliefs about one or more states in response to observing novel information about a *different* set of states

The generalization problem is the focus of much of supervised learning, and is framed as

- maximizing performance on a **test set** $D_1 = \{x_i\} \sim p(x)$
- despite only being able to see a different **train set** $D_2 = \{x_i\} \sim p(x)$

Generalization

Generalization is the process of updating your beliefs about one or more states in response to observing novel information about a *different* set of states

The generalization problem is the focus of much of supervised learning, and is framed as

- maximizing performance on a **test set** $D_1 = \{x_i\} \sim p(x)$
- despite only being able to see a different **train set** $D_2 = \{x_i\} \sim p(x)$

In SL generalization performance is optimized for

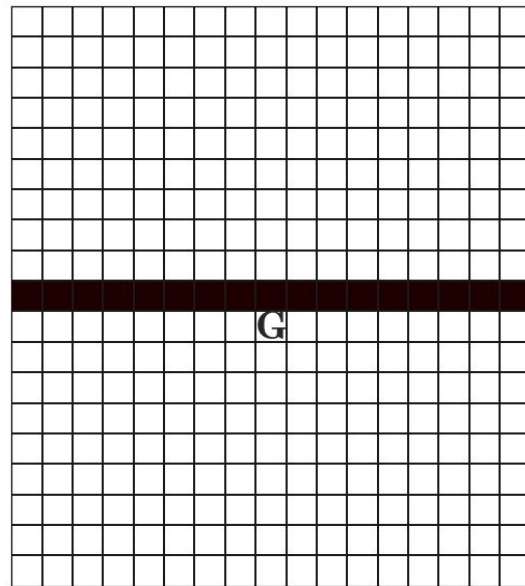
- *explicitly* (through the use of **validation** data and early stopping)
- *implicitly* (through **regularization**, via handcrafted losses or dropout)

Investigating generalization in RL

Consider the grid world on the right and lets

- estimate values for a fixed policy π
- using a neural network $v_{\theta}(s)$
- trained using SGD on the Monte-Carlo loss

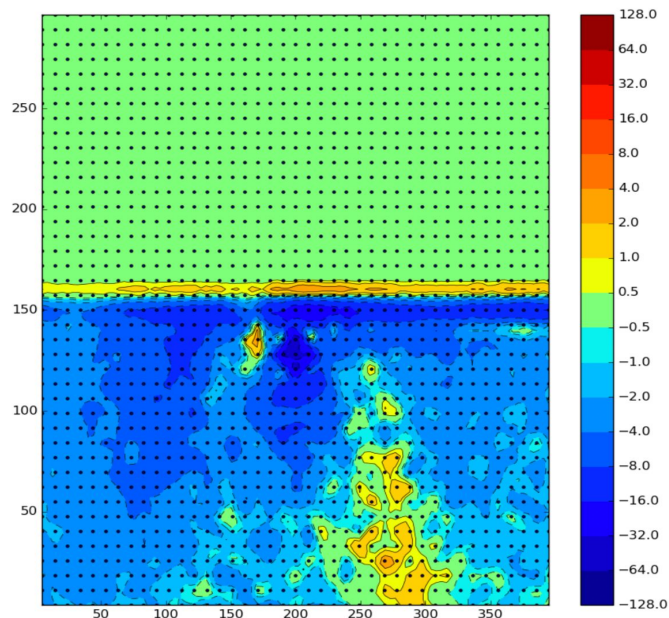
$$L = (G_t - v_{\theta}(S_t))^2$$



Inappropriate generalization: leakage

Can be difficult to predict how exactly updates to some states will end up affecting other states.

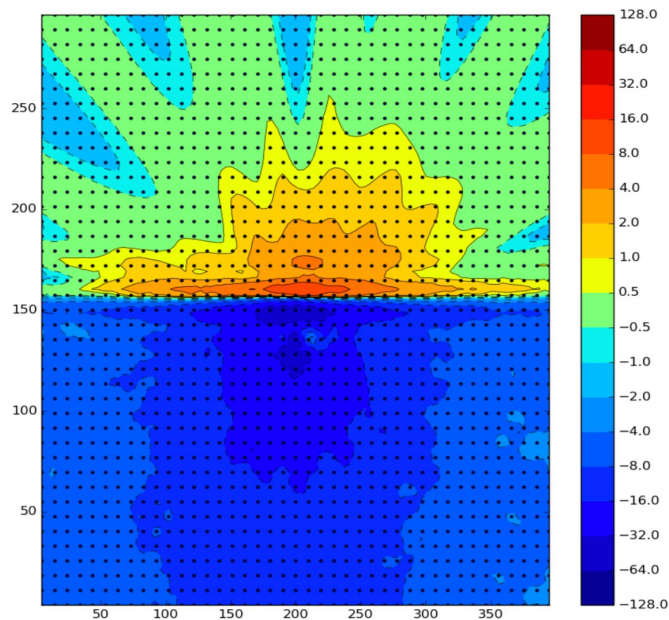
Neural network induce some degree of **smoothness** → this inductive bias is reasonable but can lead to **inappropriate generalization** when the dynamics is not smooth



Inappropriate generalization: leakage propagation

Consider the deep TD(0) algorithm instead:

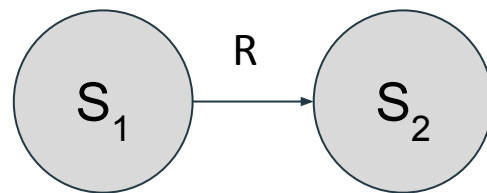
- It learns much better approximations of the values in the bottom half of the grid
- But causes **leakage propagation**, with high error across the whole of the top half



Inappropriate generalization: feedback loops

Consider the transition on the right:

If we perform deep TD update to a value network $v_{\theta}(s)$ every time we update the value of state S_1 , we will, through generalization, also perturb the values of S_2



Because we use our own estimates to construct the target $R + \gamma v_{\theta}(S_2)$ for $v_{\theta}(S_1)$

This can result in **feedback loops**!

The deadly triad

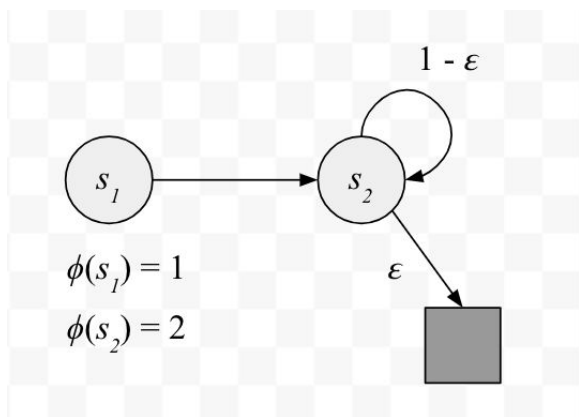
Function Approximation → Estimate values by training the parameters of some function.

Bootstrapping → Using the current agent's value estimates to construct the targets used to update those same agent's estimates.

Off-Policy → Compute learning updates from a distribution that differs from that arises under the policy whose values are being estimated and the given environment dynamics.

A simple example

A very simple example of to understand the issue was provided by Tsitsiklis and Van Roy



Simplest possible parametrization of the value

$$v_w(s) = w\phi(s)$$

Correct solution is trivially:

$$w^* = 0$$

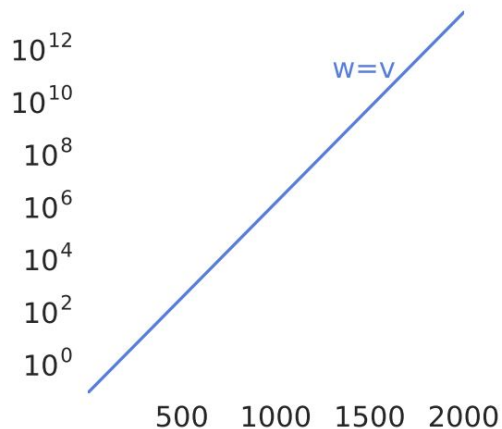
How does DQN work?

Function Approximation → DQN uses neural networks to approximate values.

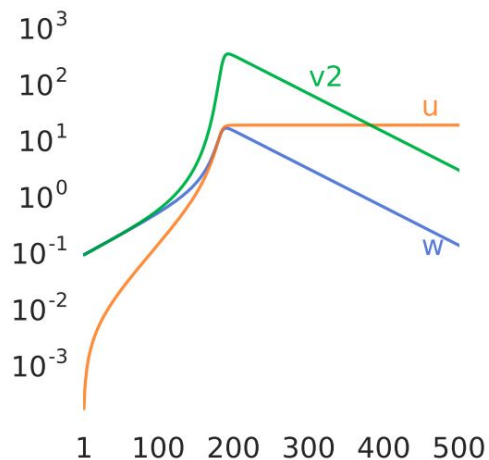
Bootstrapping → DQN uses Q-learning with bootstrap targets $R_t + \gamma \max_a Q(S_{t+1}, a)$

Off-Policy → DQN uses Q-learning, which is off-policy by construction

The specifics matter!



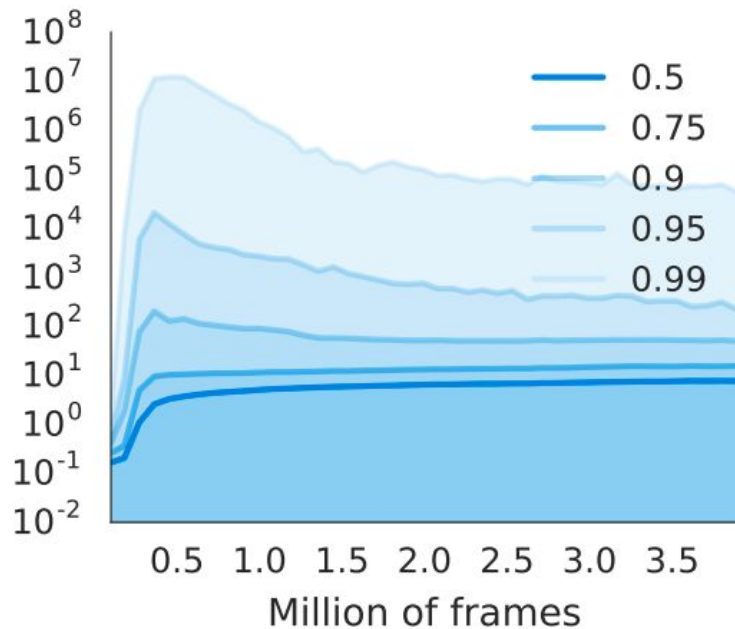
(b) $v(s) = w\phi(s)$ diverges.



(c) $v(s) = w(\phi(s) + u)$ converges.

Soft-divergence

This form of **soft-divergence** where values grow to unreasonably high values but then converge back to sensible estimates, is more common in practice than unbounded divergence.



Target Networks

Target Networks are a common remedy to this type of inappropriate generalization

We can keep **two copies** of the neural network used to estimate Q values:

- The parameters of one are updated at each step,
- The parameters of the other are a slow copy of the first one.

This reduces the opportunity for feedback loops.

Multi-step deep Q-learning

Using our own estimates Q_θ to construct targets $R_t + \gamma \max_a Q_\theta(S_{t+1}, A_{t+1})$ is called **bootstrapping**.

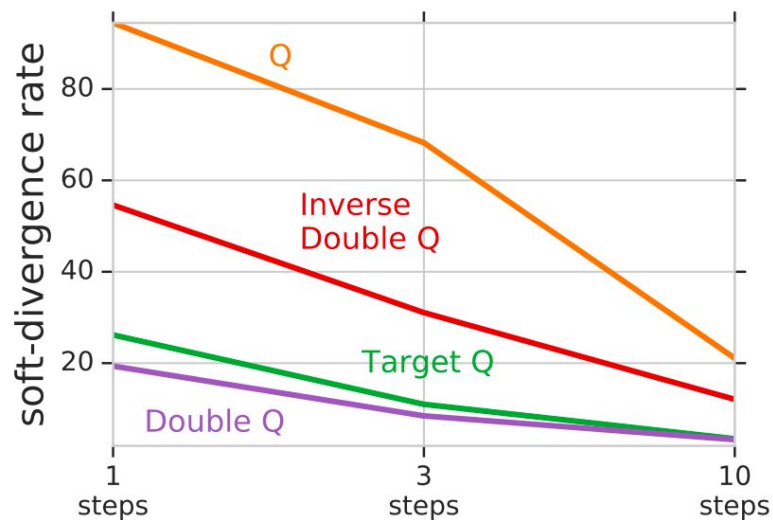
We don't however need to bootstrap necessarily after a single step.

We can construct **multi-step targets**:

$$R_t + \gamma R_{t+1} + \dots + \gamma^{K-1} R_{t+K-1} + \gamma^K \max_a Q_\theta(S_{t+K}, A_{t+K})$$

Multi-step targets and soft-divergence

multi-step returns **reduce** the reliance of our updates on bootstrapping, and the likelihood of observing divergence

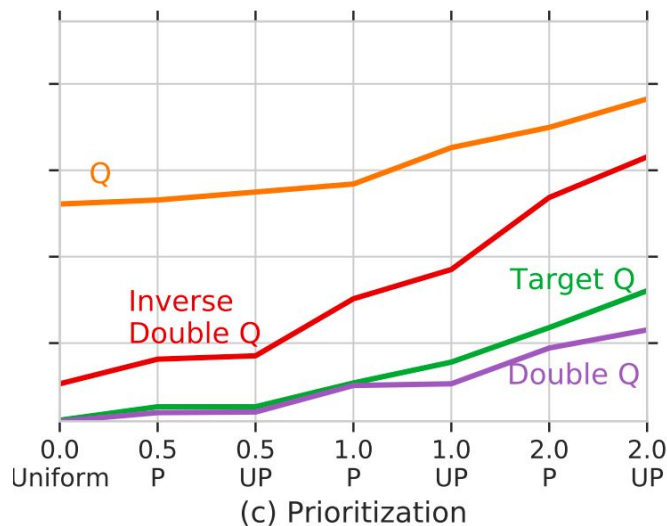


(a) Multi-step returns

Prioritized replay and soft-divergence

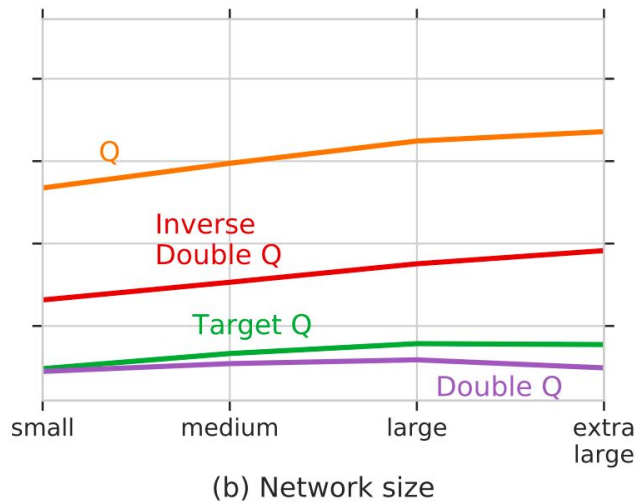
Prioritized replay makes the data distribution more off-policy, and therefore increases the likelihood of divergence.

→ importance sampling can help mitigate the effect of prioritization



Network size and soft-divergence

The likelihood of observing divergence is often higher for **larger neural networks**



Model-based RL and the deadly triad

Model-based RL can also trigger the deadly triad:

Given a learned environment model, consider the model-based prediction algorithm:

```
initialize  $v_\theta(s)$ 
while True:
     $S_t, A_t \leftarrow \text{memory.sample}()$ 
     $R_t, S_{t+1} \leftarrow \text{model.query}(S, A)$ 
     $\Delta\theta = (R_t + \gamma v_\theta(S_{t+1}) - v_\theta(S_t)) \nabla_\theta v_\theta(S_t)$ 
```

This also results in divergence if the state distribution in memory doesn't match the model's state distribution

3. State representation

Learning about many things

Learning about many things at once can be a powerful way of building better and more robust representations when using powerful Deep Learning models.

Learning about many things

Learning about many things at once can be a powerful way of building better and more robust representations when using powerful Deep Learning models.

In deep RL this is sometimes done by defining **pseudo reward** signals on top of the usual stream of experience, and predicting/controlling these signal as **auxiliary tasks**.

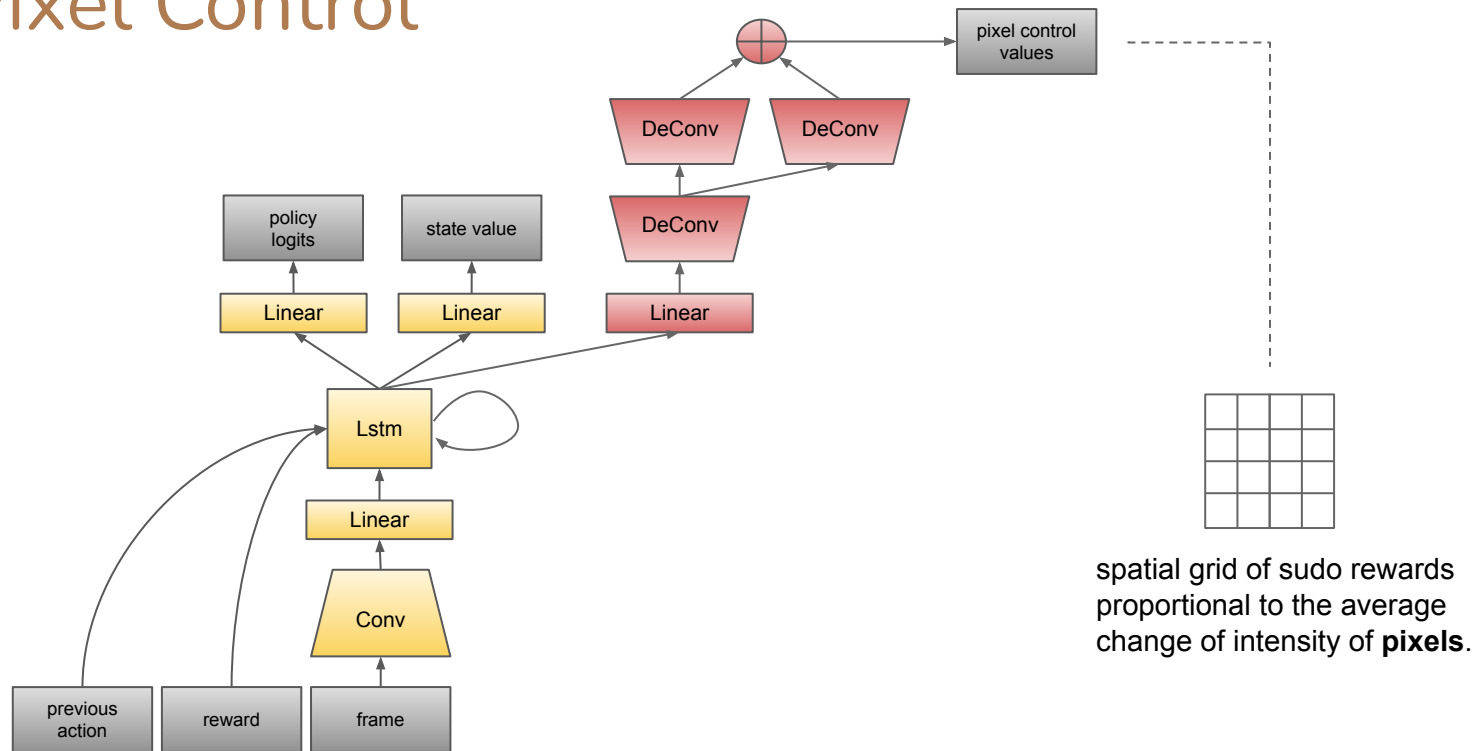
Learning about many things

Learning about many things at once can be a powerful way of building better and more robust representations when using powerful Deep Learning models.

In deep RL this is sometimes done by defining **pseudo reward** signals on top of the usual stream of experience, and predicting/controlling these signal as **auxiliary tasks**.

These auxiliary tasks can be unsupervised and exploit the rich **structure** present in the stream of observations, even before the agent manages to collect its first reward.

Pixel Control

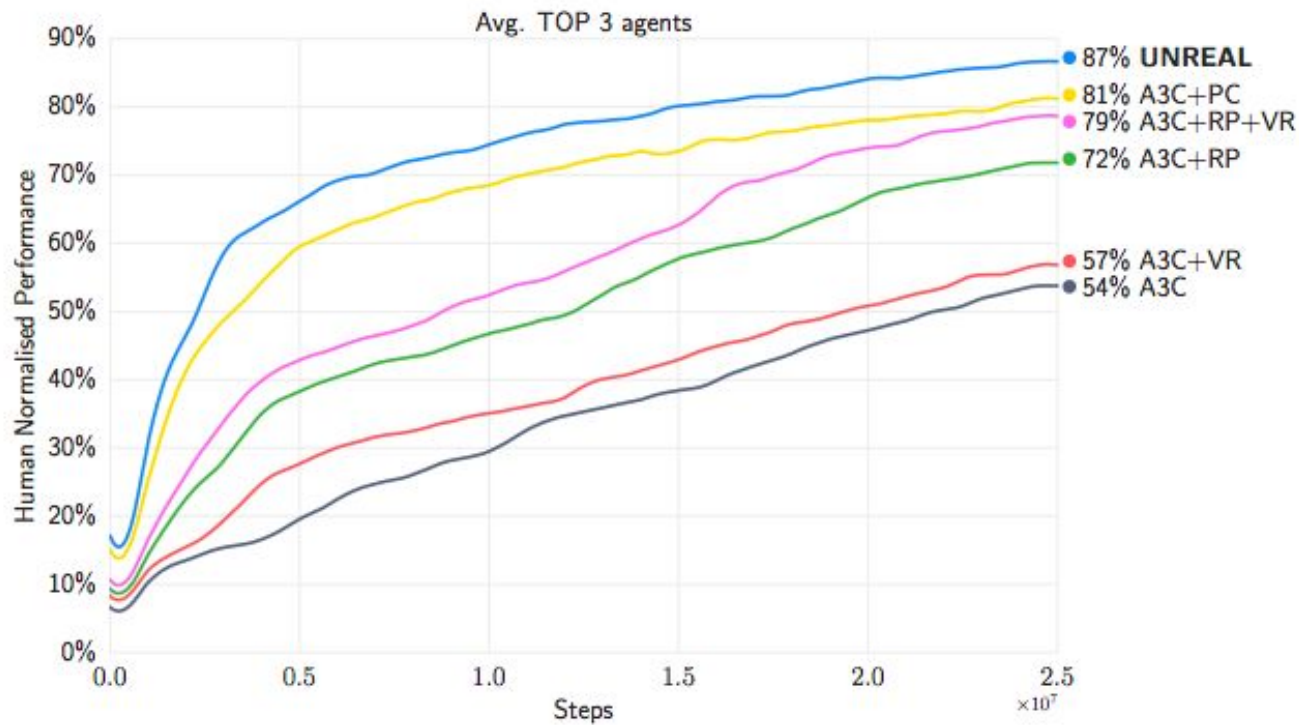


Other auxiliary tasks

There are many other ways we can make our learning signal richer

- Immediate reward prediction
- Next state prediction
- Learn at multiple time-scales (predict values for different discounts)

Labyrinth Performance



Multi-Task learning in Deep RL

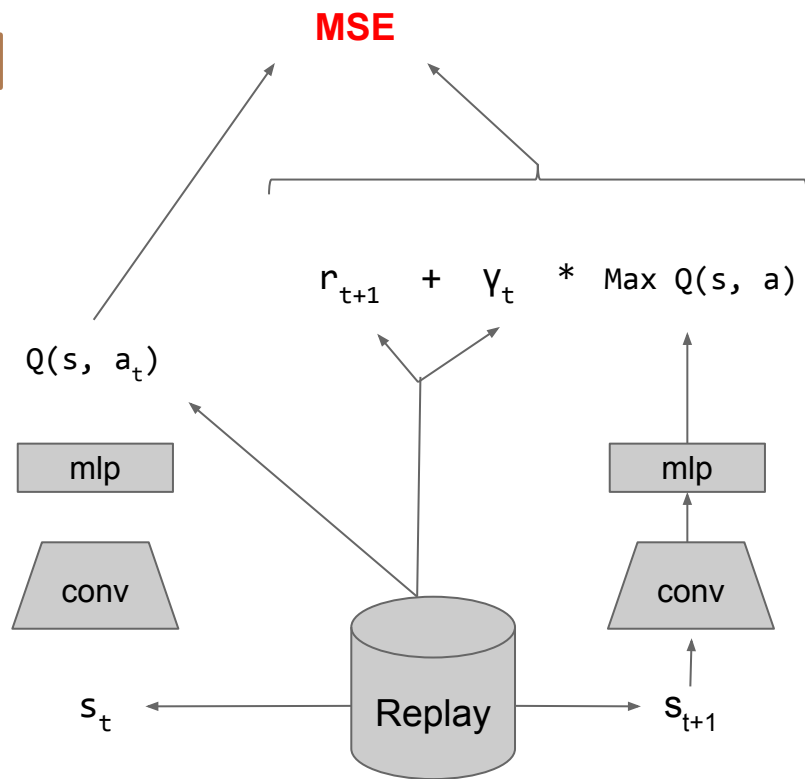
Curriculum learning

Some of the tasks could be easier or focus on specific skills that you need to master to solve the more complex ones!

Deep Exploration

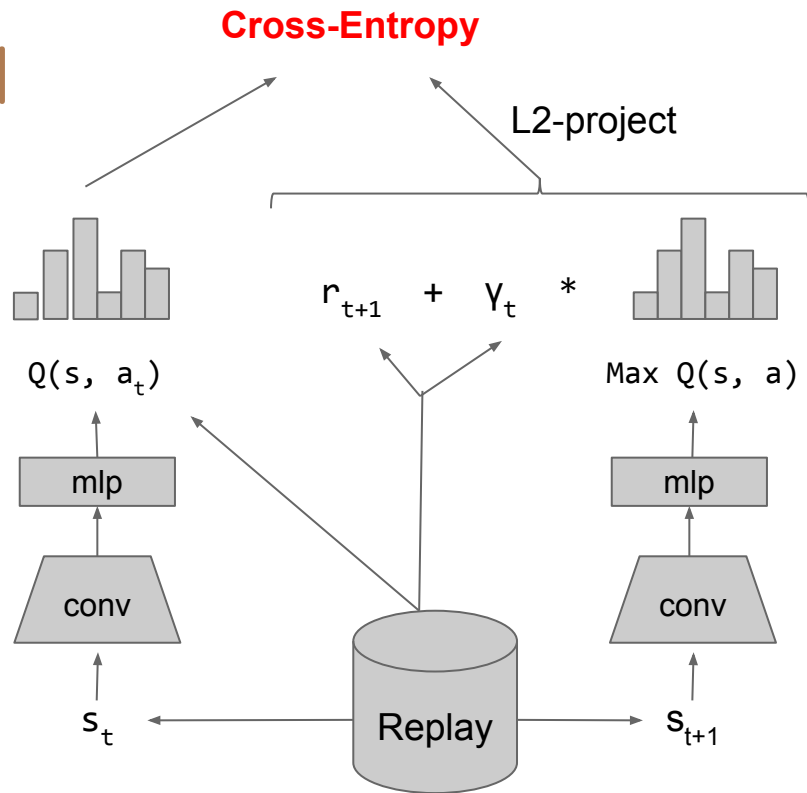
We can get deep and much more meaningful exploration by executing policies which are good for other tasks

Distrib. DQN



Distrib. DQN

- Well scaled loss
- Richer signal
- Multi-modal



3. Scaling up Deep RL

Scaling up resources

Deep Learning

More Data + More Compute → Better Performance

Scaling up resources

Deep Learning

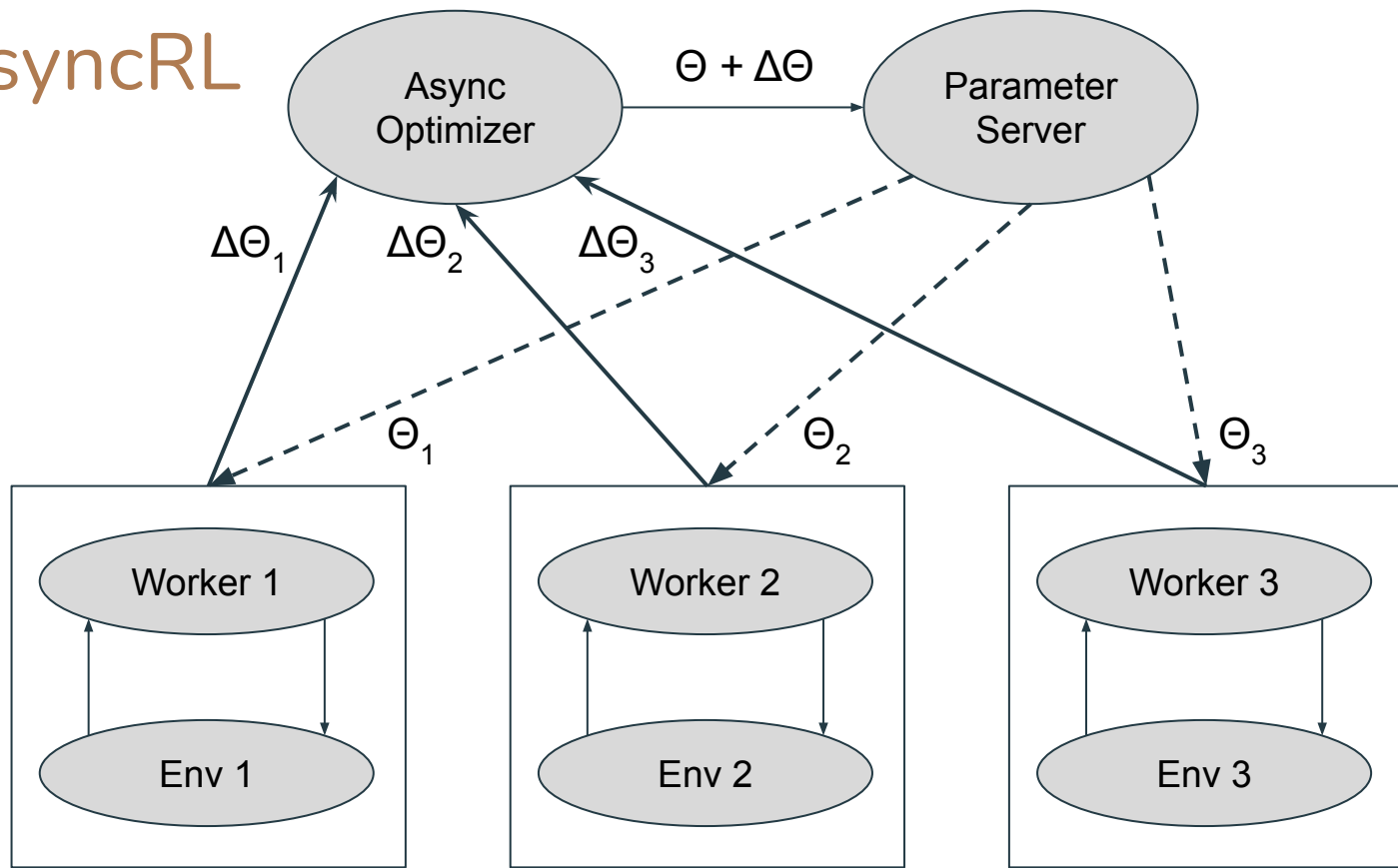
More Data + More Compute → Better Performance

Does a similar principle hold for Deep Reinforcement Learning?

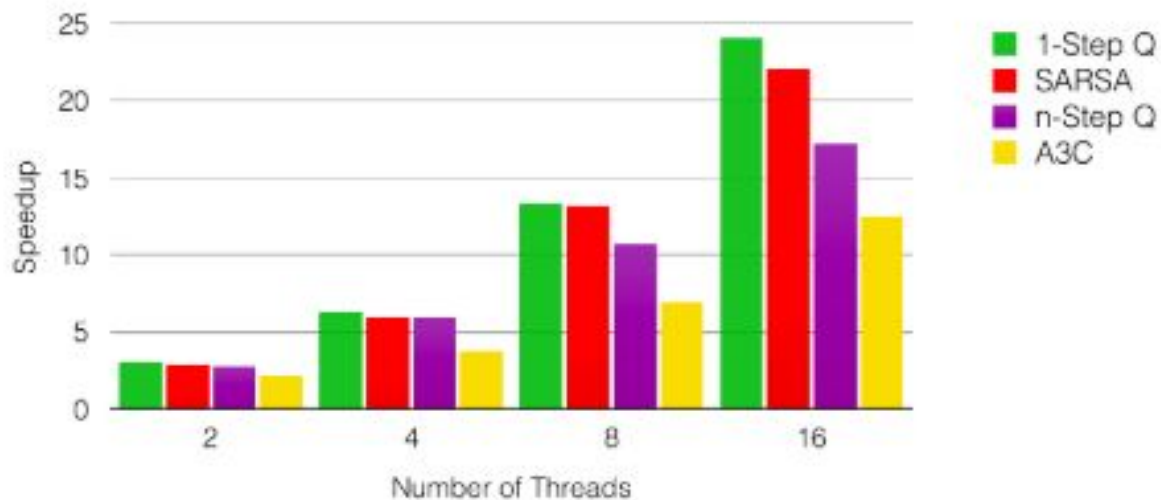
How can we leverage more compute to solve extremely complex RL problems faster?

- Multi-threading?
- GPUs?
- Distributed Computing?

AsyncRL

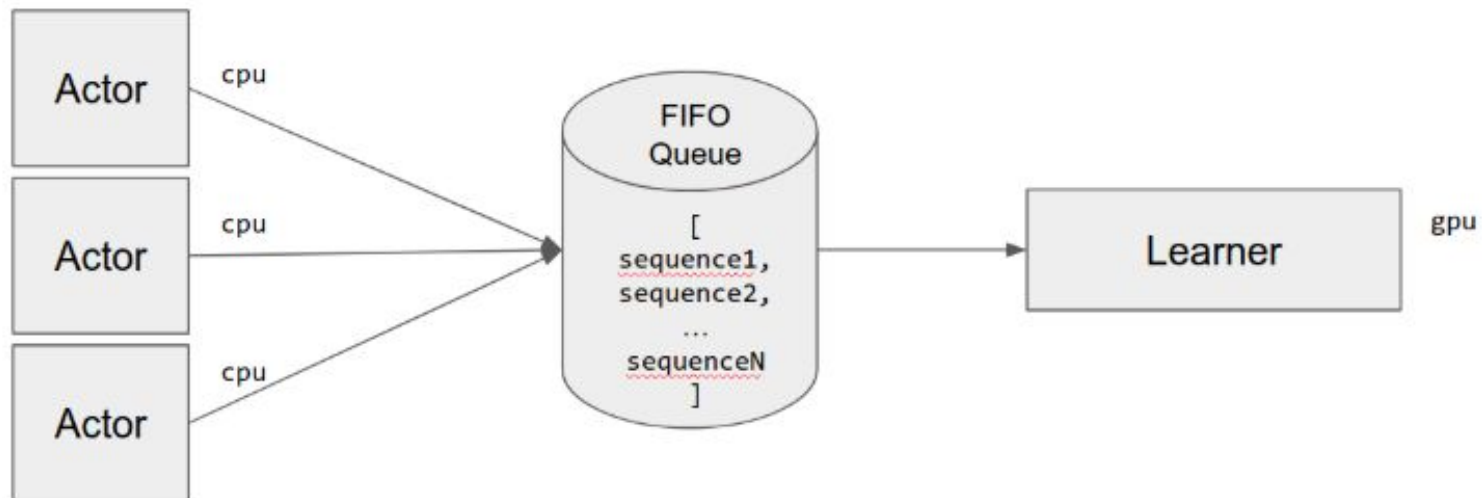


AsyncRL

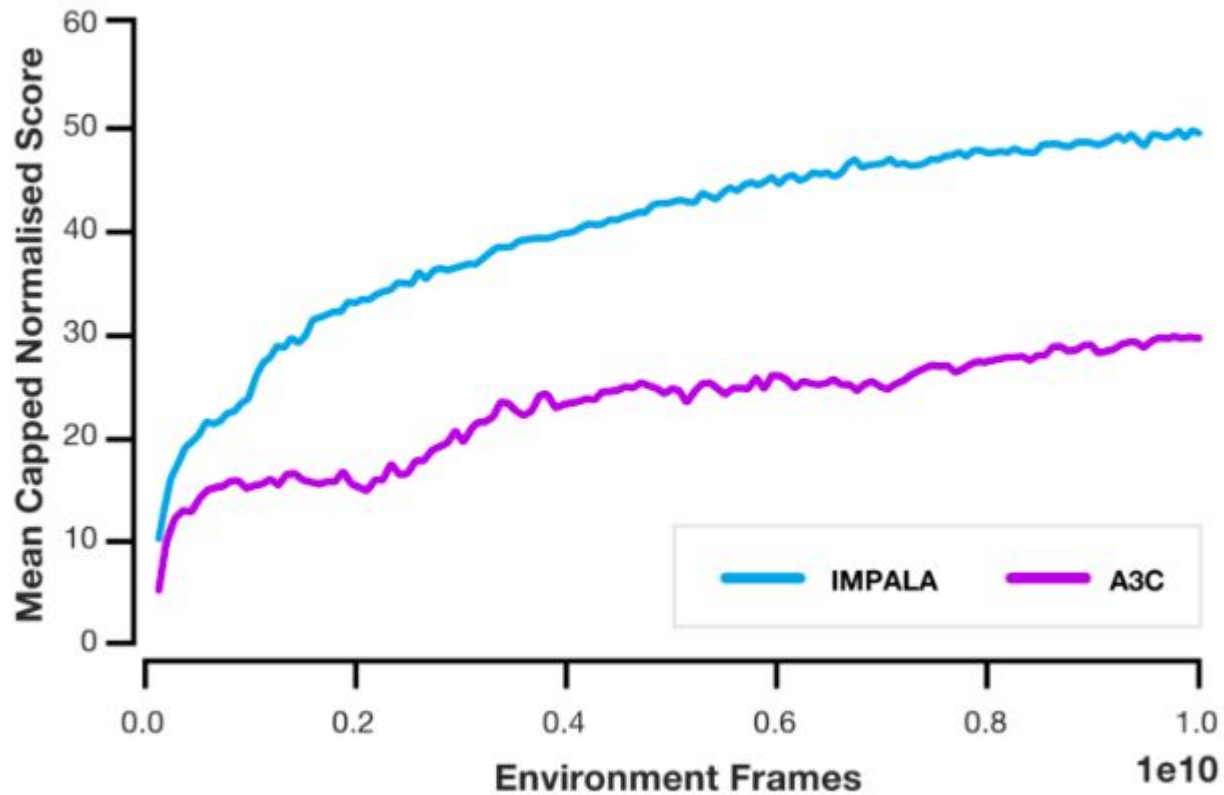


Credits: van Hasselt et al.

IMPALA



Credits: Espeholt et al.



The End